A TARIFF -AWARE APPLIANCE-LEVEL ELECTRICITY COST PREDICTION SYSTEM

A PROJECT REPORT

Submitted by

JANANI N 810022104095

KAYALVIZHI S 810022104102

in partial fulfillment for the award of the degree

of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING

UNIVERSITY COLLEGE OF ENGINEERING -BIT CAMPUS

TIRUCHIRAPPALLI

ANNA UNIVERSITY :: CHENNAI 600 025

MAY 2026

ANNA UNIVERSITY : CHENNAI 600 025

BONAFIDE CERTIFICATE

Certified that this project report **A TARIFF-AWARE APPLIANCE-LEVEL ELECTRICITY COST PREDICTION SYSTEM** is the bonafide work of **JANANI N (810022104095) , KAYALVIZHI S (810022104102)** who carried out the project work under my supervision.

SIGNATURE

DR. G. ANNAPOORANI,

HEAD OF THE DEPARTMENT,

Assistant professor (Sl.Gr.),

Department of CSE,

University College Of Engineering,

Anna University-BIT Campus,

Tiruchirappalli-620 024.

SIGNATURE

DR.C.USHARANI,

Assistant professor,

Department of CSE,

University College Of Engineering,

Anna University-BIT Campus,

Tiruchirappalli-620 024

Certified that JANANI N (810022104095) and KAYALVIZHI S (810022104102)

Were examined in a Viva – Voce examination held on _____

Internal Examiner **External Examiner**

DECLARATION

We hereby declare that the work entitled " **A TARIFF -AWARE APPLIANCE-LEVEL ELECTRICITY COST PREDICTION SYSTEM**" is submitted in partial fulfilment of the requirements for the award of the degree in B.E. (Computer Science and Engineering), in University College of Engineering, BIT Campus, Anna University, Tiruchirappalli. It is the record of my own work carried out during the academic year 2025 – 2026 under the supervision and guidance of **Dr. C.Usharani** Assistant Professor, Department of Computer Science and Engineering, University College of Engineering, BIT Campus, Anna University, Tiruchirappalli. The extent and source of information are derived from the existing literature and have been indicated through the dissertation at the appropriate places. The matter embodied in this work is original and has not been submitted for the award of any other degree, either in this or any other university.

JANANI N (810022104095)

KAYALVIZHI S (810022104102)

I certify that the declaration made above by the candidate is true.

Signature of the Guide
Dr. C. USHARANI ,
Assistant Professor,
Department of CSE,
University College of
Engineering, BIT Campus,
Anna University,
Tiruchirappalli – 620024

ACKNOWLEDGEMENT

We are deeply grateful to our Honourable Dean, **Dr. T. Senthilkumar**, Professor, Department of Automobile Engineering, UCE, BIT Campus, Anna University, for providing all the necessary resources and support that facilitated the smooth progress and completion of our Final Year Project.

We extend our heartfelt thanks to **Dr. G. Annapoorani,** Assistant Professor (Sl.Gr.) and Head of the Department of Computer Science and Engineering, for her expert guidance, thoughtful suggestions, and continued encouragement, which were instrumental in the successful completion of this work.

A special note of appreciation goes to guide come Project Coordinator **C.Usharani** , Assistant Professor, Department of Computer Science and Engineering, for her steadfast support, invaluable feedback, and dedicated mentoring throughout every phase of this project.

We also extend our sincere gratitude to the Teaching and Non-Teaching Staff and Lab Assistants of the Department of Computer Science and Engineering for their timely technical support and guidance throughout the project. Above all, we are deeply grateful to our beloved parents and siblings for their unconditional love, constant prayers, and endless encouragement, which remained our greatest strength throughout this journey.

Finally, we warmly thank our dear friends for their support, motivation, and companionship, which made this journey truly memorable and enjoyable.

ABSTRACT

The proposed A TARIFF AWARE APPLIANCE-LEVEL ELECTRICITY COST PREDICTION SYSTEM is designed to monitor, analyze, and manage the energy consumption of household appliances using real-time IoT-based data acquisition. The system integrates smart plug technology for continuous monitoring of electrical parameters such as power consumption and energy usage at the appliance level. It provides a more detailed and accurate approach compared to traditional energy meters that only measure total household consumption. Real-time data is collected using IoT-enabled smart plugs and transmitted to a centralized backend system developed using Flask. The collected data is processed, stored in an SQLite database, and used for generating historical insights such as daily and monthly energy consumption. The system applies predefined tariff slab structures to calculate electricity cost automatically based on energy usage. A RESTful API architecture enables communication between the backend, web interface, and mobile application. The frontend dashboard and Android mobile application allow users to visualize energy consumption trends, monitor appliance-wise usage, and analyze electricity costs in real time. This improves user awareness and encourages efficient energy usage. The system also helps in identifying high energy-consuming appliances, enabling users to take corrective actions to reduce electricity wastage and optimize power consumption. The proposed solution is scalable, efficient, and suitable for smart home environments. It demonstrates how IoT and web technologies can be combined to build an intelligent energy management system. Future enhancements may include machine learning-based prediction, automation of appliance control, and cloud-based large-scale energy analytics.

சுருக்கம்

இந்த முன்மொழியப்பட்ட மின்சார கட்டண அடிப்படையிலான சாதன நிலை மின்சார செலவு கணக்கீட்டு அமைப்பு என்பது வீட்டில் பயன்படும் மின்சாதனங்களின் மின்சார பயன்பாட்டை கண்காணித்து, பகுப்பாய்வு செய்து, மேலாண்மை செய்ய உருவாக்கப்பட்டுள்ளது. இந்த அமைப்பு பொருட்கள் இணையத் தொழில்நுட்பத்தை பயன்படுத்தி, நுண்ணறிவு மின்சார இணைப்பு சாதனத்தின் மூலம் ஒவ்வொரு மின்சாதனத்தின் மின்சார பயன்பாட்டை நேரடியாக கண்காணிக்கிறது.பாரம்பரிய மின்சார மீட்டர்கள் மொத்த வீட்டு மின்சார பயன்பாட்டை மட்டுமே அளவிடுகின்றன. ஆனால், இந்த அமைப்பு ஒவ்வொரு மின்சாதனத்திற்கும் தனித்தனியாக கண்காணிப்பு வழங்குகிறது. இதன் மூலம் எந்த சாதனம் அதிக மின்சாரம் பயன்படுத்துகிறது என்பதை எளிதாக அறிய முடிகிறது.மின்சாதனத்திலிருந்து சேகரிக்கப்படும் தகவல்கள் மைய செயலாக்க அமைப்பிற்கு அனுப்பப்படுகின்றன. அங்கு அவை செயலாக்கப்பட்டு தரவுத்தளத்தில் ஒழுங்காக சேமிக்கப்படுகின்றன. இந்த தகவல்கள் தினசரி மற்றும் மாதாந்திர மின்சார பயன்பாட்டை பகுப்பாய்வு செய்ய உதவுகின்றன.மேலும், முன்கூட்டியே நிர்ணயிக்கப்பட்ட கட்டண நிலை முறையின் அடிப்படையில் மின்சார செலவு தானாக கணக்கிடப்படுகிறது. மின்சார பயன்பாட்டின் அளவுக்கு ஏற்ப வெவ்வேறு கட்டணங்கள் பயன்படுத்தப்பட்டு சரியான செலவு கணக்கிடப்படுகிறது.பயனர் இடைமுகத்தின் மூலம், பயனர்கள் தங்களது மின்சார பயன்பாட்டை நேரடியாகக் காணலாம், சாதன வாரியான பயன்பாட்டை பகுப்பாய்வு செய்யலாம் மற்றும் மின்சார செலவினத்தை அறியலாம். மேலும், மொபைல் பயன்பாட்டின் மூலம் எங்கிருந்தும் கண்காணிக்கவும், மின்சாதனங்களை இயக்கவும் அல்லது நிறுத்தவும் முடியும். இந்த அமைப்பு பயனர்களுக்கு மின்சார பயன்பாட்டை பற்றிய விழிப்புணர்வை அதிகரிக்கிறது. மேலும் அதிக மின்சாரம் பயன்படுத்தும் சாதனங்களை அடையாளம் காண உதவுகிறது. இதன் மூலம் தேவையற்ற மின்சார வீணாவதை குறைத்து, மின்சாரத்தை திறமையாக பயன்படுத்த உதவுகிறது.இந்த அமைப்பு விரிவாக்கத்திற்குத் தகுந்தது, செயல்திறன் மிகுந்தது மற்றும் நவீன வீட்டு சூழல்களுக்கு மிகவும் பொருத்தமானது. எதிர்காலத்தில் செயற்கை நுண்ணறிவு அடிப்படையிலான முன்கணிப்பு, தானியங்கி கட்டுப்பாடு மற்றும் மேம்பட்ட தரவு பகுப்பாய்வு போன்ற அம்சங்களை இணைத்து மேலும் மேம்படுத்த முடியும்.

TABLE OF CONTENTS

CHAPTER NO	TITLE	PAGE NO
	ABSTRACT	V
	LIST OF TABLES	IX
	LIST OF FIGURES	X
	LIST OF ABBREVATIONS	XI
1	**INTRODUCTION**	1
	1.1 Background	1
	1.2 Problem Statement	2
	1.3 Objectives	2
2	**LITERATURE SURVEY**	3
	2.1 Introduction	3
	2.2 Review Of Existing Systems	3
	2.3 Comparative Analysis	8
3	**PROPOSED SYSTEM**	9
	3.1 Existing System	9
	3.2 Proposed System	10
	3.3 System Architecture	10
	3.4 Methodology	14
	3.4.1 Steps (Working Procedure)	14
	3.4.2 Flowchart Explanation	14
	3.5 Module Description	17
	3.5.1 Introduction	17
	3.5.2 Data Collection Module	17
	3.5.3 Data Processing &Cost Calculation Module	18

	3.5.4 Data Storage Module	18
	3.5.5 Api Module	18
	3.5.6 User Interface Module	18
4	**IMPLEMENTATION & RESULT**	19
	4.1 Tools & Technologies Used	19
	4.1.1 Hardware Requirements	19
	4.1.2 Software Requirements	20
	4.2 System Implementation	20
	4.3 Working Model	29
	4.4 Results & Outputs	29
	4.5 Testing	30
	4.6 Performance Analysis	33
5	**CONCLUSION**	34
	5.1 Conclusion	34
	5.2 Future Enhancements	34
	APPENDICES	35
	REFERENCE	46

LIST OF TABLES

TABLE NO	TITLE	PAGE NO
1	Comparative table	8
2	Hardware Requirements	19
3	Software Requirements	20
4	Tariff Slab Structure	24
5	System Output Parameters and Description	30
6	Unit Testing Result	31
7	Test Cases and Results	32
8	Comparison With Existing System	33

LIST OF FIGURES

FIGURE NO	TITLE	PAGE NO
1	Block Diagram	11
2	Flowchart Diagram	15
3	Module Architecture Diagram	17
4	Tapo Smart Plug image	21
5	Connection Diagram	22
6	Database Screen	23
7	Appliance Selection Interface	26
8	Mobile Application-Real-Time Monitoring Screen	27
9	Daily and Monthly Cost Screen	28

LIST OF ABBREVIATIONS AND NOMENCLATURE

ABBREVIATIONS

ABBREVIATION	MEANING
IOT	INTERNET OF THINGS
API	APPLICATION PROGRAMMING INTERFACE
UI	USER INTERFACE
KWH	KILOWATT-HOUR
W	WATT
V	VOLTAGE
I	CURRENT
DB	DATABASE
IP	INTERNET PROTOCOL
PDF	PORTABLE DOCUMENT FORMAT
FLASK	PYTHON WEB FRAMEWORK
SQLITE	LIGHTWEIGHT DATABASE SYSTEM

NOMENCLATURE (FORMULAS USED)

SYMBOL	DESCRIPTION
E	ENERGY (kwh)
P	POWER (kW or W)
t	TIME (hours)
V	VOLTAGE (Volts)
I	CURRENT (Amperes)

CHAPTER I

INTRODUCTION

1. INTRODUCTION

The Tariff Aware Appliance Level Electricity Cost Prediction System is designed to monitor, analyze, and estimate the electricity cost of household appliances using real-time energy data. In conventional electricity billing systems, users receive only the total energy consumption and overall cost at the end of the billing cycle. These systems do not provide detailed insights into how individual appliances contribute to energy usage and cost.The proposed system uses IoT-enabled smart plugs to collect real-time power and energy data from connected appliances. The collected data is processed using a Flask-based backend and stored in a SQLite database. Based on real-time and previously stored usage data, the system calculates electricity cost using predefined tariff slab rates. The term "prediction" in this system refers to estimating electricity cost based on current consumption trends and tariff rules, rather than using machine learning or external datasets.The system provides real-time monitoring, historical analysis, and remote control of appliances through a mobile application. By offering appliance-level monitoring and tariff-aware cost estimation, the system helps users understand their energy consumption patterns, identify high power-consuming devices, and reduce electricity wastage. This makes the system suitable for smart home energy management. The system is designed to be cost-effective, scalable, and suitable for real-time smart home applications.

1.1 BACKGROUND

The increasing use of electrical appliances in modern households has led to a significant rise in electricity consumption and cost. Traditional electricity monitoring systems provide only the total energy usage and do not offer detailed information about individual appliance consumption. This lack of transparency makes it difficult for users to manage their energy usage effectively. With the development of Internet of Things (IoT) technology, devices can now be connected and monitored in real time. Smart energy monitoring systems allow users to track appliance-level energy consumption and gain better control over their electricity usage. Tariff-aware systems further improve this process by calculating electricity cost based on predefined slab rates, which vary depending on the level of consumption.The importance of this domain lies in promoting energy efficiency, reducing electricity bills, and enabling better decision-making. By combining IoT technology with tariff-based cost analysis, the system provides a practical and efficient solution for monitoring and controlling energy consumption.

1.2 PROBLEM STATEMENT

Traditional electricity monitoring systems have several limitations that reduce their effectiveness in managing energy consumption. These systems provide only the total energy usage of a household and do not offer a detailed breakdown at the appliance level. As a result, users are unable to identify which specific appliances consume more energy or contribute significantly to their electricity bill. Another major limitation is the lack of real-time monitoring, which prevents users from tracking their energy usage instantly and taking immediate actions to reduce unnecessary consumption. In addition, existing systems do not support tariff-aware cost calculation during usage, and electricity cost is typically known only at the end of the billing cycle without any prior estimation. Furthermore, most traditional systems do not provide remote control functionality, making it impossible for users to turn appliances ON or OFF from a remote location. These limitations clearly indicate the need for an advanced system that offers real-time monitoring, appliance-level analysis, tariff-aware cost estimation, and remote control capabilities.

1.3 OBJECTIVES

The primary objective of the proposed system is to monitor the real-time energy consumption of household appliances using IoT-enabled smart plugs. The system provides detailed appliance-level analysis, allowing users to understand how each device contributes to overall energy usage.

Another important objective is to estimate electricity cost based on predefined tariff slab rates and current energy consumption. This helps users track their electricity expenses in real time and manage their usage more efficiently.

In addition, the system aims to enable remote ON/OFF control of appliances through a mobile application. It also stores historical energy data for analysis and provides a user-friendly interface for visualizing energy usage patterns and cost trends.

CHAPTER 2
LITERATURE SURVEY

2.1 INTRODUCTION

Literature review provides an overview of existing research and technologies related to energy monitoring and appliance-level electricity analysis. It helps in understanding different approaches such as machine learning, Non-Intrusive Load Monitoring (NILM), and IoT-based systems used in smart energy management.

Many existing systems focus on predicting energy consumption, disaggregating total load into appliance-level data, and analyzing usage patterns. However, most of these systems rely on datasets, complex machine learning models, or indirect measurement techniques. These approaches often require high computational resources and are difficult to implement in real-time applications.The proposed system differs by using IoT-enabled smart plugs for direct measurement and tariff-based cost estimation, providing a simple, real-time, and practical solution for smart homes.

2.2 REVIEW OF EXISTING SYSTEMS / PAPERS

1. A TEMPORAL-SPATIAL GRAPH NETWORK WITH A LEARNABLE ADJACENCY MATRIX FOR APPLIANCE-LEVEL ELECTRICITY CONSUMPTION PREDICTION

AUTHOR: DANDAN LI ET AL., 2025

This paper proposes a Temporal-Spatial Graph Neural Network (TS-GNN) for predicting appliance-level electricity consumption in smart environments. The system models each appliance as a node in a graph structure, where relationships between appliances are represented as edges. A key innovation in this work is the learnable adjacency matrix, which allows the model to automatically learn hidden dependencies between appliances without manual configuration. The model captures both temporal patterns (changes in energy usage over time) and spatial relationships (interaction between different appliances). Temporal graph convolution techniques are applied to process sequential energy data effectively. The system is trained using historical electricity consumption datasets to improve prediction accuracy. It is particularly useful in smart home energy forecasting and demand management systems. The model can achieve high accuracy in predicting future energy usage patterns. However, it requires large datasets for training and validation. The computational complexity is very high due to graph-based processing. It is not suitable for low-cost IoT systems or real-time lightweight applications. The implementation requires advanced machine learning knowledge and significant processing power. Despite its accuracy, practical deployment in simple systems is limited.

Limitations

- Requires large historical datasets
- High computational complexity
- Complex implementation and Not suitable for real-time low-cost systems

2. REAL-TIME ENERGY MONITORING USING IoT AND CLOUD COMPUTING

AUTHOR: K. SURESH ET AL., 2021

This paper presents an IoT-based real-time energy monitoring system integrated with cloud computing for efficient data storage and analysis. The system uses smart sensors and IoT devices to collect energy consumption data from household appliances. The collected data is transmitted to a cloud server where it is processed and stored. Users can access the data through web or mobile applications, enabling remote monitoring. The system also supports visualization using graphs and dashboards to understand energy usage patterns. It helps in improving energy efficiency by providing real-time insights. The use of cloud technology allows scalability and storage of large datasets. However, the system mainly focuses on monitoring and does not include detailed tariff-based cost calculation. It also depends heavily on internet connectivity and cloud services. Data privacy and security are important concerns. Therefore, while the system is useful for monitoring, it lacks cost prediction features required for practical billing systems.

Limitations:

- No detailed tariff-based cost calculation
- Depends on cloud and internet connectivity and Data privacy concerns

3. IoT-BASED SMART ENERGY MANAGEMENT SYSTEM FOR HOME AUTOMATION

AUTHOR: R. KUMAR ET AL., 2022

This paper proposes an IoT-based smart energy management system that integrates home automation with energy monitoring. The system uses smart plugs and sensors to collect real-time energy data from appliances. It allows users to monitor energy consumption through a mobile application. The system also supports remote control of appliances, enabling users to turn devices ON or OFF from anywhere. It helps reduce energy wastage and improves efficiency. The system is easy to implement and suitable for small-scale applications. It provides real-time updates and user-friendly interfaces. However, it lacks advanced data analysis and does not provide detailed cost calculation based on tariff slabs. The system also does not maintain long-term historical data effectively. Security of IoT devices remains a concern. Therefore, while useful for automation and monitoring, it lacks advanced analytical features.

Limitations:

- No tariff-based billing system
- Limited historical data analysis
- Security issues in IoT devices

4. MACHINE LEARNING-BASED ENERGY CONSUMPTION PREDICTION IN SMART HOMES

AUTHOR: J. LI ET AL., 2023

This paper focuses on predicting energy consumption using machine learning techniques in smart home environments. The system uses historical data collected from IoT devices to train predictive models such as regression and neural networks. It analyzes usage patterns and forecasts future energy consumption. This helps users in planning and reducing electricity usage. The model improves accuracy by learning complex patterns in data. It is useful for energy forecasting and smart grid applications. However, the system requires large amounts of historical data for training. It involves complex algorithms and high computational resources. Real-time implementation is difficult due to processing requirements. The system also does not directly provide appliance-level monitoring using smart plugs. Therefore, it is suitable for prediction but not for simple real-time monitoring systems.

Limitations:

- Requires large training datasets
- High computational complexity
- Not suitable for real-time IoT systems

5. EDGE COMPUTING-BASED SMART ENERGY MONITORING SYSTEM

AUTHOR: Y. ZHANG ET AL., 2024

This paper introduces an edge computing-based smart energy monitoring system for real-time data processing. Instead of sending data to the cloud, the system processes energy data locally using edge devices. This reduces latency and improves real-time performance. The system collects data from IoT devices and performs analysis at the edge level. It provides faster response and reduces dependency on internet connectivity. The system is efficient for real-time applications and improves data privacy. However, it requires specialized hardware for edge processing. The implementation cost is higher compared to simple systems. It also lacks detailed tariff-based cost calculation and user-friendly interfaces. Maintenance of edge devices can be complex. Therefore, while efficient, it is not suitable for low-cost smart home applications.

Limitations:

- Requires specialized hardware
- Higher implementation cost
- Limited cost calculation features

6. SMART ENERGY MONITORING SYSTEM USING IoT AND MOBILE APPLICATION

AUTHOR: P. SHARMA ET AL., 2020

This paper presents an IoT-based smart energy monitoring system integrated with a mobile application for real-time monitoring of electricity consumption. The system uses smart sensors and IoT devices to collect electrical parameters such as voltage, current, and power from household appliances. The collected data is transmitted to a

backend server and displayed on a mobile application, allowing users to monitor energy usage anytime. The system also supports graphical visualization of data, helping users understand usage trends. It improves user awareness and encourages energy-saving

behavior. The mobile application provides a simple and user-friendly interface for monitoring. However, the system mainly focuses on monitoring and does not include detailed cost calculation based on tariff slabs. It also lacks advanced analytics such as appliance ranking and prediction. The system depends on internet connectivity for data transmission. Therefore, while effective for monitoring, it does not fully support cost management.

Limitations:

- No tariff-based cost calculation
- Limited data analysis features
- Depends on internet connectivity

7. IoT-BASED ELECTRICITY MONITORING AND BILLING SYSTEM

AUTHOR: S. VERMA ET AL., 2022

This paper proposes an IoT-based electricity monitoring and billing system that automates energy measurement and billing. The system collects energy consumption data using smart meters and sends it to a server for processing. It calculates electricity cost based on usage and generates bills automatically. The system reduces manual effort in billing and improves accuracy. It provides users with access to their consumption data through a web interface. The system helps in efficient energy management and billing transparency. However, it focuses on total household consumption rather than appliance-level monitoring. It does not provide

detailed insights into individual appliance usage. The system also lacks real-time control features such as ON/OFF switching of devices. Therefore, while useful for billing, it does not support detailed monitoring and control.

Limitations:

- No appliance-level monitoring
- Limited real-time control and Less detailed analysis

8. AI-BASED SMART HOME ENERGY MANAGEMENT SYSTEM

AUTHOR: L. WANG ET AL., 2023

This paper introduces an AI-based smart home energy management system that uses artificial intelligence to optimize energy usage. The system collects data from IoT devices and applies machine learning algorithms to analyze consumption patterns. It predicts future energy usage and suggests optimal appliance usage schedules. The system helps reduce electricity consumption and improves efficiency. It also supports automation features for smart homes. However, the implementation requires large datasets and complex algorithms. It involves high computational cost and requires powerful hardware. The system is difficult to implement in real-time low-cost environments. It also does not focus on simple tariff-based cost calculation. Therefore, while advanced, it is not suitable for basic IoT monitoring systems.

Limitations:

- Complex implementation
- Requires large datasets and High computational cost

9.REAL-TIME SMART ENERGY METERING USING IoT AND DATA ANALYTICS

AUTHOR: M. ALI ET AL., 2024

This paper presents a real-time smart energy metering system using IoT and data analytics techniques. The system collects energy data from smart meters and processes it to provide real-time monitoring and analysis. It uses data analytics tools to generate insights such as consumption trends and peak usage times. The system helps users manage energy efficiently and reduce costs. It supports visualization through dashboards and reports. However, the system focuses mainly on aggregated energy data rather than appliance-level monitoring. It does not provide detailed tariff-based cost calculation. The implementation may require advanced analytics tools and infrastructure. Therefore, while useful for analysis, it lacks detailed appliance-level features required for smart plug-based systems.

Limitations:

- No appliance-level monitoring
- Limited tariff calculation
- Requires analytics tools

2.3 COMPARITIVE ANALYSIS TABLE

Paper No	Mehod Used	Tools/Technology Used	Accuracy/ Performance	Merits	Demerits
Paper 1 (2025)	Temporal -Spatial Graph Neural Network (TS-GNN)	Deep Learning, Graph Neural Network	Very High	High prediction accuracy, captures appliance relationships	Requires large dataset, high complexity, not suitable for real-time
Paper 2 (2021)	IoT + Cloud Monitoring	IoT Sensors, Cloud Computing	Moderate	Real-time monitoring, scalable storage	No tariff calculation, depends on internet, privacy issues
Paper 3 (2022)	IoT-based Home Automation	Smart Plug, Mobile App	Moderate	Remote control, easy implementation	No cost calculation, limited analysis
Paper 4 (2023)	Machine Learning Prediction	Regression, Neural Networks	High	Accurate prediction, pattern analysis	Requires large data, high computation, not real-time
Paper 5 (2024)	Edge Computing System	Edge Devices, IoT	High	Low latency, faster processing	Expensive hardware, complex setup
Paper 6 (2020)	IoT + Mobile Monitoring	Sensors, Mobile App	Moderate	Real-time monitoring, user-friendly	No tariff system, limited features
Paper 7 (2022)	IoT Billing System	Smart Meter, Server	High	Automatic billing, accurate measurement	No appliance-level monitoring
Paper 8 (2023)	AI-based Energy Management	AI/ML Algorithms	Very High	Smart optimization, efficient usage	Complex, high cost, not suitable for simple systems
Paper 9 (2024)	IoT + Data Analytics	Analytics Tools, IoT	High	Trend analysis, visualization	No appliance-level tracking, limited tariff logic

CHAPTER 3
PROPOSED SYSTEM

3.1 EXISTING SYSTEM

The existing system for electricity monitoring in residential and commercial environments mainly relies on conventional energy meters that are installed at the main power supply point. These meters are designed to measure the total electricity consumption of the entire building rather than individual appliances. The data collected by these meters is typically used for monthly billing purposes and does not provide detailed insights into how electricity is consumed by different devices. Even with the advancement of smart meters, the information provided is still limited to overall energy usage and basic consumption patterns. Users are unable to identify which appliances consume more power, making it difficult to manage and optimize energy usage effectively. There is no provision for appliance-level monitoring, which is essential for understanding energy distribution within a household. In addition, existing systems do not support real-time monitoring, meaning users cannot view live energy consumption or respond immediately to sudden increases in power usage. The lack of real-time data reduces the ability to take preventive actions to minimize electricity wastage. Another major drawback is the absence of remote control features, as users cannot operate appliances such as turning them ON or OFF through any digital platform. This limits automation and convenience in modern smart home environments. Furthermore, electricity billing in existing systems is calculated as a total value without providing a detailed breakdown based on usage patterns or tariff slabs, reducing transparency for consumers. There is also no integration with mobile applications or web-based dashboards, which restricts accessibility and user interaction. The system does not provide graphical analysis, historical comparisons, or appliance ranking based on energy consumption. Users cannot track daily or monthly trends in a meaningful way. Additionally, there is no support for identifying high energy-consuming appliances or suggesting energy-saving measures. The system is completely passive and does not provide any intelligent features or recommendations. It also lacks connectivity with IoT devices, which limits its capability to function in smart environments. Overall, the existing system is inefficient, lacks flexibility, and does not meet the requirements of modern energy management. It provides only basic functionality, making it difficult for users to control, monitor, and optimize their electricity usage effectively.

Limitations of Existing System

- Only total energy consumption is measured; no appliance-level.
- No real-time data display or monitoring.
- No remote ON/OFF control or mobile application support.
- No detailed analysis or automatic tariff-based cost calculation.

3.2 PROPOSED SYSTEM

The proposed system is a Tariff-Aware Appliance-Level Electricity Monitoring and Cost Prediction System designed to overcome the limitations of traditional electricity monitoring methods by integrating IoT technology with real-time data processing. In this system, each electrical appliance is connected to an IoT-enabled smart plug, specifically a Tapo smart plug, which continuously measures electrical parameters such as power consumption, voltage, current, and energy usage. These smart plugs act as intelligent sensing devices that not only monitor energy consumption but also allow control over the connected appliances. The collected data is transmitted through a Wi-Fi network to a Flask-based backend server, which serves as the core processing unit of the system. The backend receives real-time data, processes it to calculate meaningful values such as energy usage in kilowatt-hours, and applies a tariff-based cost calculation using predefined electricity slab rates. The processed data is then stored in an SQLite database for maintaining historical records. The system exposes RESTful APIs that enable communication between the backend and the frontend interfaces, which include a web dashboard and an Android mobile application. Through these interfaces, users can monitor real-time energy consumption, analyze usage patterns, compare appliance performance, and control appliances remotely by switching them ON or OFF. The system also supports additional features such as identifying high energy-consuming appliances and generating PDF reports for electricity usage and billing. Overall, the proposed system provides an efficient, scalable, and user-friendly solution for smart energy management in modern households.

Features of Proposed System

- Real-time energy monitoring
- Appliance-level tracking
- ON/OFF control of appliances
- Automatic tariff-based cost calculation
- Historical data storage and analysis
- Top energy-consuming appliance detection andPDF report generation
- Web and mobile application support.

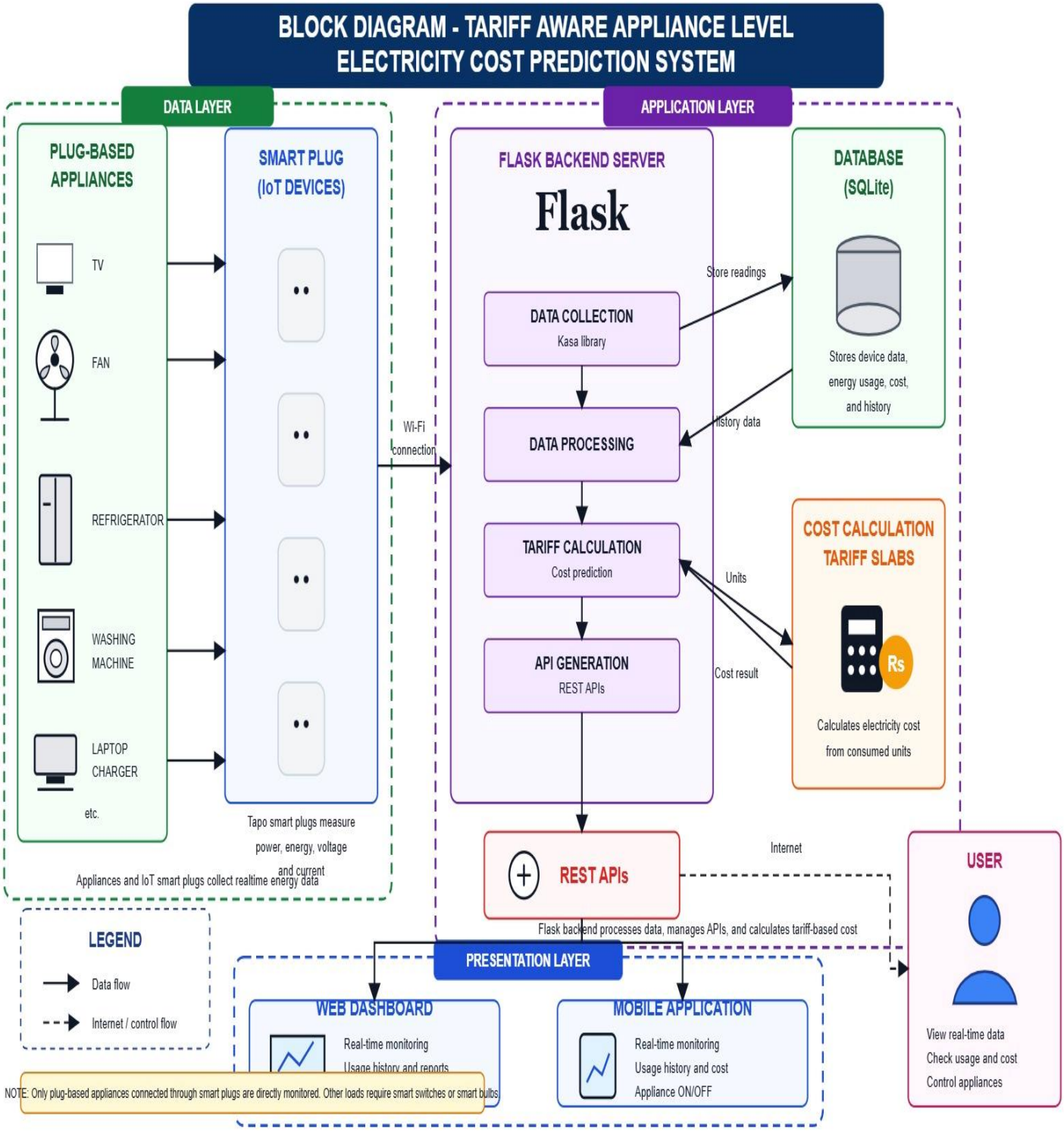
3.3 SYSTEM ARCHITECTURE

The system architecture is designed using a multi-layered approach to ensure efficient communication, processing, and data management.

The proposed system compress of three main layers namely Presentation layer, Application layer and Data layer. The presentation layer includes the web dashboard and mobile application, which provide an interface for user interaction. The application layer consists of the Flask backend server, which processes data, manages APIs, and controls communication

between components. The data layer includes the SQLite database and IoT smart plugs, which are responsible for data storage and data collection respectively.

The architecture ensures smooth data flow from appliances to the user. The smart plug collects real-time energy data and sends it to the backend server through a Wi-Fi network. The backend processes the data, calculates electricity cost, stores it in the database, and sends the results to the frontend through APIs. The frontend displays the data in an easy-to-understand format, allowing users to monitor and control appliances effectively.



1. PLUG-BASED APPLIANCES

This block represents the electrical devices used in a household, such as televisions, refrigerators, washing machines, heaters, and chargers. These appliances are the primary sources of electricity consumption in the system. In the proposed model, each appliance is connected through a smart plug, which allows individual monitoring instead of measuring only total consumption. This helps the system identify appliance-level energy usage and calculate electricity cost accurately. However, this block mainly supports plug-based appliances, while non-pluggable devices like ceiling fans or fixed lights require additional smart devices such as smart switches.

2. SMART PLUG (IoT DEVICE)

The smart plug acts as an IoT-enabled energy monitoring device. It measures electrical parameters such as voltage, current, power consumption, and energy usage of the connected appliance. The smart plug continuously collects real-time data and transmits it to the backend server. It also supports remote ON/OFF control functionality. This block plays a crucial role in enabling appliance-level monitoring and provides the raw data required for tariff-based cost calculation.

3. WI-FI NETWORK

The Wi-Fi network acts as the communication medium between the smart plug and the backend server. It enables real-time transmission of energy data from the IoT device to the system. The performance of the system depends on the reliability and speed of the network. A stable Wi-Fi connection ensures uninterrupted data flow, which is essential for real-time monitoring and cost calculation. It also allows remote access to the system from any location.

4. FLASK BACKEND SERVER

The Flask backend server acts as the central processing unit of the system. It receives real-time data from the smart plug and performs operations such as data processing, tariff-based cost calculation, database storage, and API management. The backend is developed using Python and Flask, providing a lightweight and efficient environment. It integrates all system components and ensures smooth data flow between devices and user interfaces.

5. DATA COLLECTION MODULE

The Data Collection Module retrieves real-time energy data from the smart plug using the Kasa Python library. It connects to the IoT device using IP address and authentication credentials. Once connected, it fetches parameters such as power consumption, daily energy usage, and monthly energy usage. This module ensures continuous and accurate data collection, which is essential for cost calculation.

6. DATA PROCESSING MODULE

The Data Processing Module converts raw data into meaningful information. It performs operations such as data validation, cleaning, and unit conversion (e.g., watts to kilowatt-hours). It prepares the data for tariff calculation. This module ensures that the processed data is accurate, consistent, and ready for further analysis.

7. TARIFF CALCULATION MODULE

The Tariff Calculation Module calculates electricity cost based on a slab-based tariff system. Different ranges of energy consumption are charged at different rates. For example, the first set of units may be free, while higher ranges are charged progressively. This module applies the appropriate tariff rates to the energy consumption data and calculates daily and monthly electricity cost. It is the core part of the system that makes it "tariff-aware".

8. SQLITE DATABASE

The SQLite database stores all processed data, including energy usage, cost details, appliance name, and timestamps. It maintains both real-time and historical records. The database allows efficient storage and retrieval of data, which is useful for analysis, reporting, and visualization.

9. REST API MODULE

The REST API module acts as a communication interface between the backend server and frontend applications. It provides endpoints for fetching real-time data, retrieving historical records, and sending control commands. This module ensures smooth data exchange between different parts of the system.

10. WEB DASHBOARD

The web dashboard is developed using HTML, CSS, and JavaScript. It displays real-time energy consumption, cost details, and historical data in the form of tables and graphs. It allows users to monitor appliance usage and understand electricity consumption patterns easily.

11. MOBILE APPLICATION

The mobile application provides access to the system through smartphones. It connects to the backend using APIs and displays real-time energy data and cost information. Users can also control appliances remotely using the mobile app, making the system more flexible and user-friendly.

12. USER

The user is the final component of the system who interacts with it through the web dashboard or mobile application. The user can monitor energy consumption, view cost details, analyze historical data, and control appliances remotely. The system is designed to be simple and user-friendly so that users can easily manage their electricity usage.

3.4 METHODOLOGY

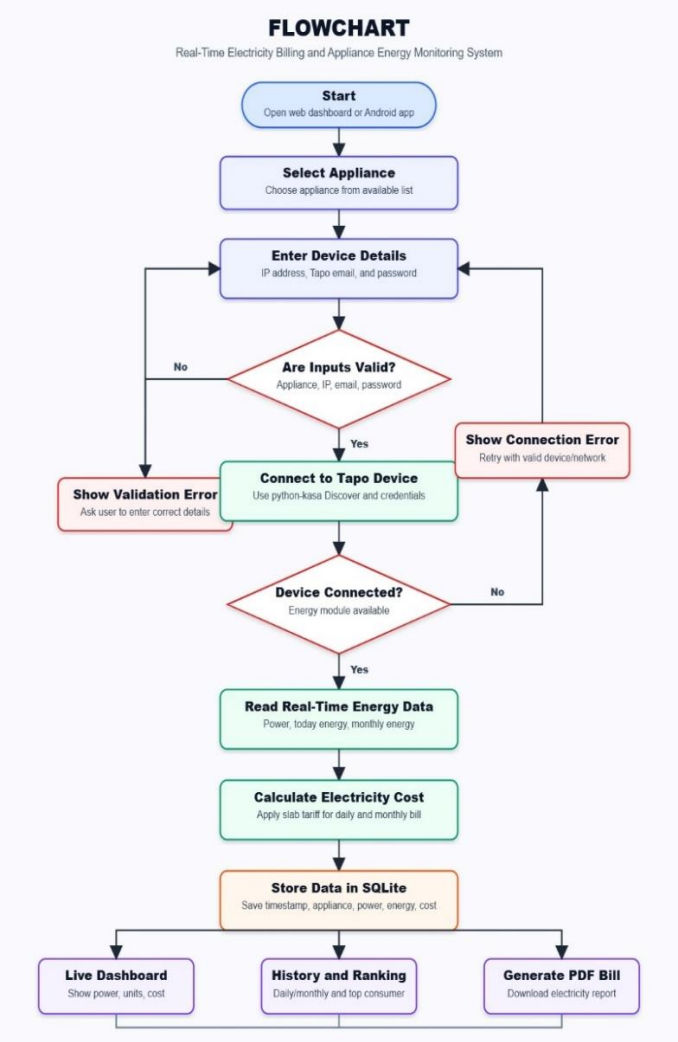
3.4.1 Steps (Working Procedure)

1. Connect electrical appliance to smart plug
2. Smart plug measures power and energy consumption
3. System connects to smart plug using Kasa library
4. Real-time data is fetched through network
5. Flask backend receives the data
6. Data is processed and validated
7. Electricity cost is calculated using tariff slabs
8. Processed data is stored in SQLite database
9. REST APIs send data to frontend
10. User views real-time data on mobile/web app
11. User can control appliance (ON/OFF)

3.4.2 FLOWCHART EXPLANATION

The flowchart represents the complete working process of the Tariff-Aware Appliance-Level Electricity Cost Prediction System. It illustrates how the system operates from start to end in a logical and structured manner. The process begins when the user connects an electrical appliance to the smart plug. The smart plug measures real-time electrical parameters such as power and energy consumption. The system then establishes a connection with the smart plug using the Kasa Python library. Once the connection is successful, real-time data is fetched through the Wi-Fi network.

The Flask backend server receives this data for further processing. The system validates and processes the data to ensure accuracy and consistency. The processed data is converted into standard units for calculation. The system then applies tariff slab rates to calculate electricity cost. The calculated cost includes daily and monthly usage details. The processed data is stored in the SQLite database for maintaining records. The backend uses REST APIs to send data to the frontend. The user can view real-time energy consumption and cost details through the web or mobile application. The system also allows remote ON/OFF control of appliances. Overall, the flowchart clearly explains the complete working and data flow of the system.



FLOWCHART DIAGRAM

Step 1: Start

The process begins when the user opens the web application or mobile application. This step initializes the system and prepares it to receive user input and start monitoring energy consumption.

Step 2: Connect Appliance to Smart Plug

In this step, the electrical appliance is connected to the IoT-enabled smart plug. The smart plug acts as a monitoring device that measures energy consumption. Without this connection, the system cannot collect appliance-level data.

Step 3: Enter Device Details

The user enters the required details such as IP address, email, and password. These details are necessary for the system to establish a secure connection with the smart plug device.

Step 4: Validate Input
The system checks whether the entered details are correct and valid. If the input is incorrect, the system displays an error message and asks the user to re-enter the details. This step ensures secure and accurate device connection.

Step 5: Connect to Smart Plug
After successful validation, the system connects to the smart plug using the Kasa Python library. If the connection fails, the system shows a connection error. If successful, the system proceeds to the next step.

Step 6: Fetch Real-Time Energy Data
Once connected, the system retrieves real-time energy data from the smart plug. This includes power consumption (Watts), daily energy usage (kWh), and monthly energy usage (kWh). This data forms the base for cost calculation.

Step 7: Process Data
The collected data is processed by the backend server. This involves cleaning the data, validating it, and converting it into standard units such as kilowatt-hours. This step ensures accuracy in further calculations.

Step 8: Apply Tariff Calculation
In this step, the system calculates electricity cost based on slab-wise tariff rates. The system applies different rates for different ranges of energy consumption and computes daily and monthly cost.

Step 9: Store Data in Database
The processed data along with cost details is stored in the SQLite database. This includes parameters such as timestamp, appliance name, energy usage, and cost. This step helps in maintaining historical records.

Step 10: Send Data via API
The backend sends the processed data to the frontend using REST APIs. These APIs allow smooth communication between the server and user interface.

Step 11: Display Data to User
The user can view real-time power consumption, energy usage, and cost details through the web dashboard or mobile application. The system may also display graphs and reports for better understanding.

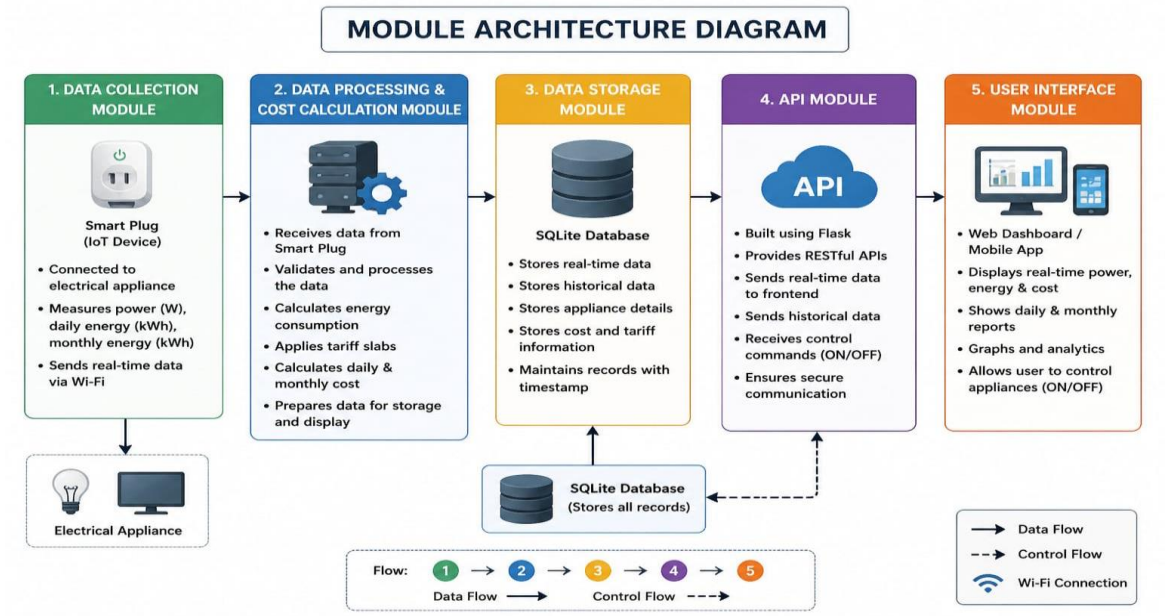
Step 12: Appliance Control (ON/OFF)
The system allows the user to control appliances remotely. The user can turn devices ON or OFF through the application, and the command is sent to the smart plug via the backend.

Step 13: Stop
The process ends when the user exits the application or stops monitoring. The system can be restarted anytime for further monitoring.

3.5 MODULE DESCRIPTION

3.5.1 INTRODUCTION

The Tariff Aware Appliance level electricity cost prediction system is designed using a modular approach to ensure better performance, scalability, and easy maintenance. Each module in the system is responsible for a specific functionality such as data collection, storage, processing, and visualization. The modules are interconnected and work together to provide real-time monitoring, cost calculation, and appliance control. This modular design makes the system flexible and allows easy modification or addition of new features without affecting the entire system.



3.5.2 DATA COLLECTION MODULE

The Data Collection Module is the first stage of the system and is responsible for gathering real-time energy data from IoT-enabled smart plugs. These smart plugs are connected to electrical appliances such as televisions, refrigerators, and washing machines. The smart plug measures important electrical parameters including power consumption (Watts), daily energy usage (kWh), and monthly energy usage (kWh). It continuously collects this data and sends it to the backend server through a Wi-Fi connection. This module ensures that accurate and real-time data is available for further processing. Without this module, the system cannot function effectively, as all calculations depend on the collected data.

3.5.3 DATA PROCESSING & COST CALCULATION MODULE

The Data Processing and Cost Calculation Module acts as the core processing unit of the system. It receives raw data from the Data Collection Module and performs several important operations. First, it validates the data to remove errors and ensure correctness. Then, it processes and converts the data into standard units suitable for analysis. After processing, the module calculates electricity cost using a slab-based tariff system. In this system, different ranges of energy consumption are charged at different rates. The module computes both daily and monthly costs based on energy usage. It also prepares the processed data for storage and visualization. This module makes the system "tariff-aware" by applying real-world electricity billing logic.

3.5.4 DATA STORAGE MODULE

The Data Storage Module is responsible for storing all processed data in an organized manner using an SQLite database. It stores information such as timestamp, appliance name, power consumption, energy usage, and calculated cost. This module maintains both real-time and historical data, which is useful for analysis and report generation. It ensures data consistency and allows efficient retrieval whenever required. The stored data can be used to generate graphs, compare usage patterns, and identify high energy-consuming appliances. This module plays a key role in maintaining system records and supporting long-term data analysis.

3.5.5 API Module

The API Module acts as a communication bridge between the backend system and the frontend applications. It is implemented using the Flask framework and provides RESTful APIs for data exchange. This module allows the frontend to request real-time data, historical records, and analysis results from the backend. It also receives control commands from the user, such as turning appliances ON or OFF, and forwards them to the smart plug. The API module ensures secure, fast, and reliable communication between all components of the system. Without this module, integration between backend and frontend would not be possible.

3.5.6 USER INTERFACE MODULE

The User Interface Module provides a platform for users to interact with the system. It includes both a web dashboard and a mobile application. This module displays real-time energy consumption, cost details, and historical data in a user-friendly format using tables, charts, and graphs. Users can easily monitor appliance usage, compare energy consumption, and understand electricity costs. Additionally, the module allows users to control appliances remotely by sending ON/OFF commands through the interface. This module enhances usability and ensures that the system is accessible anytime and anywhere.

CHAPTER 4
IMPLEMENTATION AND RESULT

4.1 TOOLS AND TECHNOLOGIES USED

4.1.1 HARDWARE REQUIREMENTS

The hardware components used in the proposed system are essential for real-time energy monitoring and control. The system requires a basic computing device, IoT hardware, and networking components to function efficiently.

The primary hardware component is the Tapo Smart Plug, which acts as an IoT-based energy monitoring device. It is connected between the power source and the electrical appliance to measure parameters such as voltage, current, power, and energy consumption. The smart plug also supports remote ON/OFF control of appliances.

A computer system or laptop with a minimum Intel Core i3 processor, 4 GB RAM, and sufficient storage is required to run the backend server and development tools. Additionally, an Android smartphone is required to run the mobile application and monitor system outputs. A stable Wi-Fi network is necessary to establish communication between the smart plug and the backend server. The overall hardware setup ensures smooth operation and real-time data transmission.

Component	**Minimum Specification**	**Recommended Specification**
Processor	Intel Core i3 (2.6GHz) or equivalent	Intel Core i5 / i7
Ram	4 GB	8GB or above
Storage	160 GB HDD	256 GB SSD
IoT Devices	Tapo Smart Plug (Basic Model)	Tapo Smart Plug with Energy Monitoring (P110)
Mobile Device	Android Smartphone (Basic)	Android Smartphone (Latest Version)
Network	Wi-Fi Connection	High-speed Wi-Fi (Stable Connection)
Operating System	Windows 10	Windows 11

4.1.2 SOFTWARE REQUIREMENTS

The software components used in the system are responsible for data processing, storage, and user interaction. The backend is developed using Python with the Flask framework, which handles communication with the IoT device, data processing, and API management.The Kasa Python library is used to connect and communicate with the Tapo smart plug, enabling real-time data retrieval. SQLite is used as the database to store energy consumption data, cost details, and historical records.

The frontend is developed using HTML, CSS, and JavaScript to create a user-friendly web dashboard. Additionally, Android Studio is used to develop the mobile application for remote monitoring and control.Development tools such as PyCharm or Visual Studio Code are used for coding and debugging. The system runs on a Windows operating system, which supports all required tools and frameworks.

COMPONENT	SOFTWARE	VERSION
Programming Language	Python	3.8 or higher
Web Framework	Flask	2.0 or higher
IoT Library	Kasa	Latest Version
Frontend	HTML,CSS,JavaScript	Latest Version
Mobile Development	Android Studio	Latest Version
Database	SQLite	3.x
IDE	PyCharm / VS Code	Latest Version
Operating System	Windows	10 / 11

4.2 SYSTEM IMPLEMENTATION

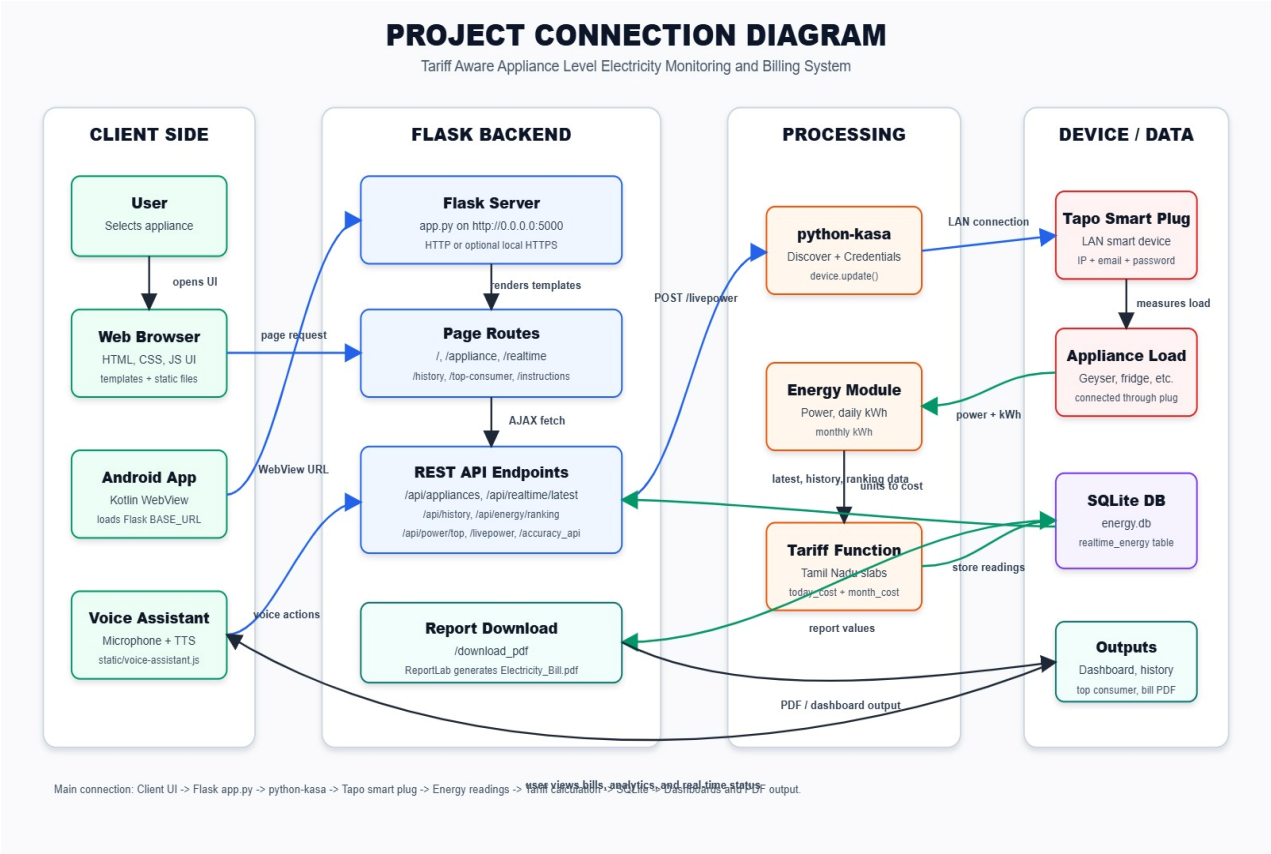
The system is implemented using a Flask-based backend that communicates with IoT smart plugs using the Kasa Python library. The backend collects real-time energy data such as power and energy consumption from the smart plug. This data is processed and stored in an SQLite database. REST APIs are developed to transfer data between the backend and frontend.
The frontend is designed using HTML, CSS, and JavaScript to display real-time data, graphs, and tables. An Android mobile application is also developed to allow users to monitor energy usage and control appliances remotely. The implementation ensures real-time data flow, accurate cost calculation, and smooth user interaction.

Tapo Smart Plug (Tapo P110)

The system uses the TP-Link Tapo P110 Smart Plug as the primary IoT device for real-time energy monitoring and remote appliance control. This smart plug is connected between the electrical power supply and the appliance, allowing it to directly measure important electrical parameters such as voltage, current, power consumption, and energy usage. By continuously collecting this data, the device provides accurate and real-time information about the energy consumption of individual appliances. The Tapo P110 is designed with a 16A rating, making it suitable for handling moderate to high-power devices like heaters, refrigerators, and washing machines. In the Indian electrical setup, where socket types vary, a 6A–16A adapter may be required for proper fitting. However, for safety and better performance, high-power appliances should ideally be connected to a dedicated 16A socket to avoid overheating and loose connections. Proper installation of the smart plug ensures reliable operation and accurate measurement of energy usage.

The smart plug connects to the local Wi-Fi network, enabling seamless communication with the backend server and mobile application. Using the Kasa Python library, the Flask-based backend retrieves real-time data from the device through its IP address. This data is processed, stored in the SQLite database, and used for generating insights such as daily and monthly energy consumption and cost estimation based on tariff slabs. In addition to monitoring, the smart plug supports remote ON/OFF control, allowing users to operate appliances from anywhere using a mobile application. It also includes scheduling features that help automate appliance usage, reducing unnecessary power consumption. The system ensures efficient data flow between the device, backend, and user interface through REST APIs. Overall, the integration of the smart plug enhances the system by providing accurate real-time monitoring, remote control capabilities, and improved energy management for users.

CONNECTION DIAGRAM



The above diagram represents the connection and working of the proposed tariff-aware appliance-level electricity monitoring and cost calculation system. On the client side, the user interacts with the system through a web browser, Android application, or voice assistant interface. The user selects an appliance and sends requests to the Flask backend server. The backend processes these requests using predefined page routes and REST API endpoints.

The Flask server communicates with the smart plug using the python-kasa library, which establishes a connection through the local Wi-Fi network. The smart plug measures real-time electrical parameters such as power consumption and energy usage from the connected appliance. This data is then sent to the energy module in the backend, where it is processed and converted into meaningful values such as daily and monthly energy consumption.

The tariff function module calculates the electricity cost based on slab-based pricing rules. The processed data, including energy usage and cost, is stored in the SQLite database for future reference. The backend then sends this data to the frontend through APIs, where it is displayed in the form of dashboards, history reports, and graphical analysis.

Additionally, the system supports report generation, where users can download electricity bills in PDF format. The overall system ensures real-time monitoring, accurate cost calculation, and remote control of appliances through an integrated and efficient architecture. The system also supports a voice assistant (Jarvis), which allows users to control appliances using voice commands through the frontend interface.

DATABASE (SQLite)

DB Browser for SQLite - C:\Users\Administrator\Desktop\eb_bill_project\energy.db

File Edit View Tools Help

New Database | Open Database | Write Changes | Revert Changes | Undo | Open Project | Save Project

Database Structure | Browse Data | Edit Pragmas | Execute SQL

Table: realtime_energy | Filter in any column

	id	timestamp	power	today_energy	month_energy	today_cost	month_cost	appliance
	Filter	Filter	Filter	Filter	Filter	Filter	Filter	Filter
1	1	2026-03-18 15:28:28.537405	71.346	0.014	0.014	0.0	0.0	LaptopCharger
2	2	2026-03-18 15:28:33.377708	72.162	0.014	0.014	0.0	0.0	LaptopCharger
3	3	2026-03-18 15:28:38.169114	73.01	0.014	0.014	0.0	0.0	LaptopCharger
4	4	2026-03-18 15:28:43.224025	67.76	0.014	0.014	0.0	0.0	LaptopCharger
5	5	2026-03-18 15:28:48.180040	72.198	0.014	0.014	0.0	0.0	LaptopCharger
6	6	2026-03-18 15:28:53.097712	73.348	0.014	0.014	0.0	0.0	LaptopCharger
7	7	2026-03-18 15:28:58.096176	67.752	0.014	0.014	0.0	0.0	LaptopCharger
8	8	2026-03-18 15:29:03.088136	69.569	0.014	0.014	0.0	0.0	LaptopCharger
9	9	2026-03-18 15:29:08.169225	71.979	0.014	0.014	0.0	0.0	LaptopCharger
10	10	2026-03-18 15:29:13.211367	73.349	0.014	0.014	0.0	0.0	LaptopCharger
11	11	2026-03-18 15:29:18.083065	69.218	0.015	0.015	0.0	0.0	LaptopCharger
12	12	2026-03-18 15:29:23.106850	71.036	0.015	0.015	0.0	0.0	LaptopCharger
13	13	2026-03-18 15:29:28.107649	59.098	0.015	0.015	0.0	0.0	LaptopCharger
14	14	2026-03-18 15:29:33.105037	68.287	0.015	0.015	0.0	0.0	LaptopCharger
15	15	2026-03-18 15:29:38.325322	73.679	0.015	0.015	0.0	0.0	LaptopCharger
16	16	2026-03-18 15:29:44.957300	69.161	0.015	0.015	0.0	0.0	LaptopCharger
17	17	2026-03-18 15:29:48.288284	71.919	0.015	0.015	0.0	0.0	LaptopCharger
18	18	2026-03-18 15:29:53.112583	69.655	0.015	0.015	0.0	0.0	LaptopCharger

The above screen shows the SQLite database used in the proposed Smart Appliance Electricity Monitoring System for storing all energy-related information collected from IoT-enabled smart plugs. The database acts as the central storage unit where all processed data from the backend is saved in a structured format for future retrieval, analysis, and reporting.

The database contains a table (for example, realtime_energy) which stores important fields such as timestamp, appliance name, power consumption (in watts), energy usage (kWh), and calculated cost based on tariff slabs. Each record represents real-time or historical energy usage of a specific appliance at a particular time.When the system fetches data from the smart plug, the Flask backend processes it and automatically inserts the values into the database. This ensures that both current and historical energy consumption data are maintained without manual intervention.

The stored data is later used by different modules of the system, such as the web dashboard and mobile application, to display graphs, generate reports, and analyze usage patterns. It also helps in identifying high energy-consuming appliances and calculating monthly electricity bills accurately.Overall, the SQLite database plays a crucial role in ensuring data persistence, structured storage, and efficient retrieval, making the system reliable and suitable for real-time energy monitoring and analysis.

TARIFF CALCULATION MODULE

The tariff calculation module is responsible for computing the electricity cost based on energy consumption. The system uses a slab-based tariff model, where different ranges of energy units are charged at different rates. This ensures accurate and realistic cost calculation similar to real-world electricity billing systems.

After receiving energy data from the smart plug, the backend converts the usage into kilowatt-hours (kWh). The tariff function then applies predefined slab rates to calculate the cost. For example, initial units may have lower charges, while higher consumption falls into higher tariff slabs with increased cost per unit.

The calculation is performed automatically in the Flask backend using a Python function. The system calculates both daily and monthly electricity costs based on real-time data. The computed values are stored in the SQLite database and displayed to the user through the web dashboard and mobile application.

This module plays a crucial role in helping users understand their electricity expenses and manage energy usage efficiently. It eliminates the need for manual calculation and ensures accurate billing based on consumption patterns.

Tariff Slab Structure

Units Range	Rate (₹ per unit)
0 -100	0 (Free)
101-200	₹2.25
201-400	₹4.50
401-500	₹6.00
501-600	₹8.00
601-800	₹9.00
801-1000	₹10.00
Above 1000	₹11.00

Example: Full Calculation for 750 Units

Let us calculate the electricity bill for 750 units step by step.

Step 1: 0 – 100 units

Free of cost

100 X 0 = 0

Step 2: 101 – 200 unit

100 X 2.25 = 225

Step 3: 201 – 400 units

100 X 4.50 = 900

Step 4: 401 – 500 units

100 X 6 = 600

Step 5: 501 – 600 units

100 X 8 = 800

Step 6: 601 – 750 units

Remaining = 150 units

150 X 9 = 1350

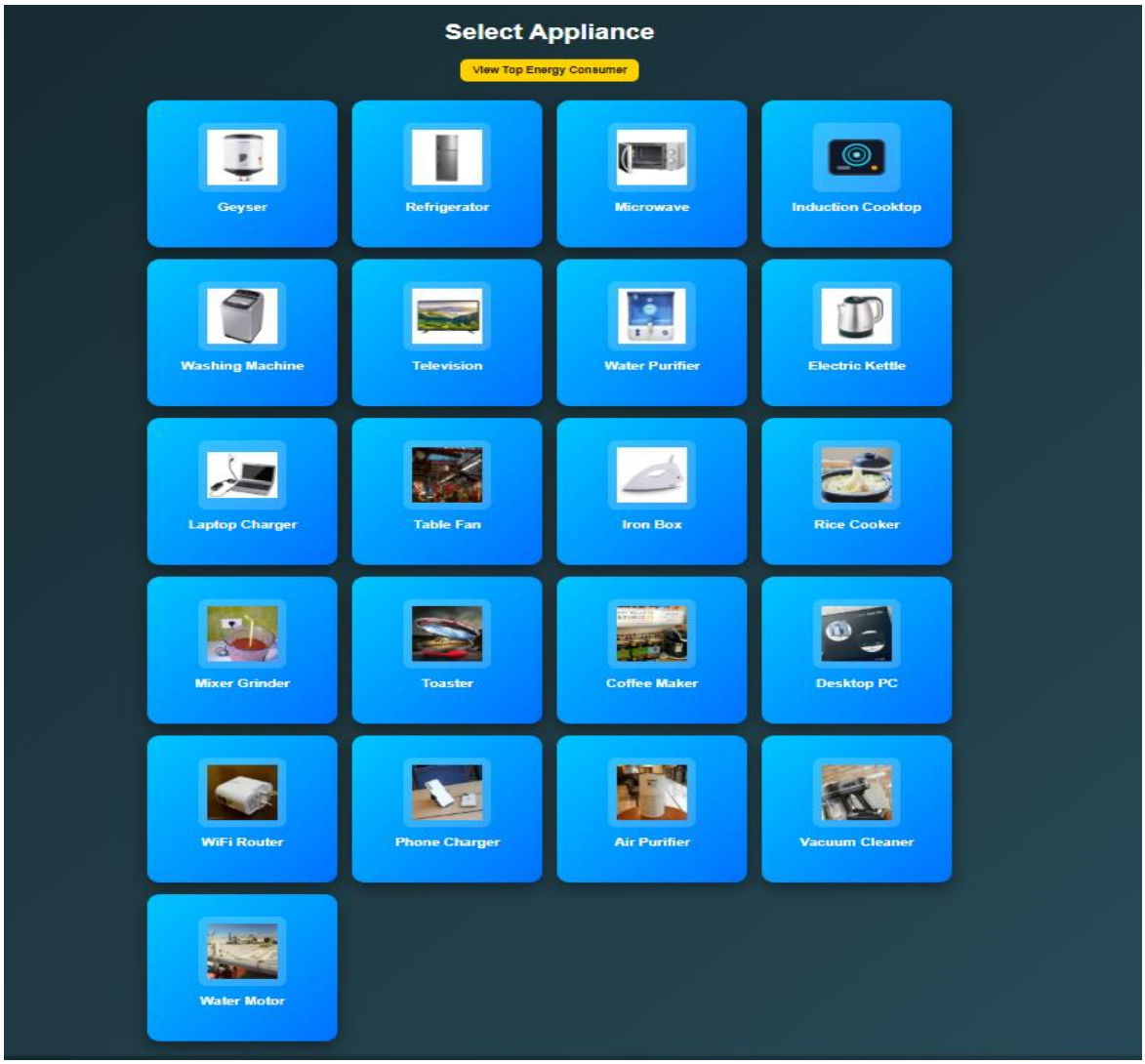
Total Cost Calculation **Total Cost = 0 + 225 + 900 + 600 + 800 + 1350**

Total Cost = ₹3875

Final Result

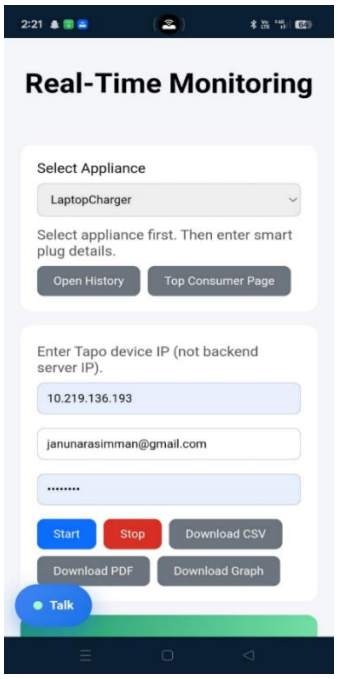
For 750 units, the total electricity cost is**: ₹3875**

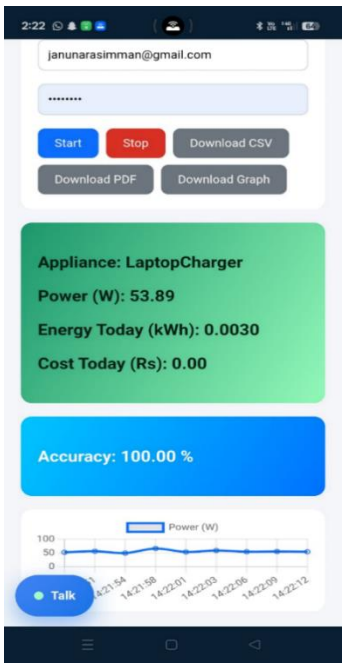
APPLIANCE SELECTION INTERFACE



The above screen represents the appliance selection interface of the system. It displays a list of electrical appliances such as geyser, refrigerator, microwave, washing machine, and others. The user can select any appliance to monitor its energy consumption. This interface provides an easy and user-friendly way to choose devices for real-time monitoring. It also includes an option to view the top energy-consuming appliance, which helps users identify high energy usage devices. This module improves usability and allows efficient management of multiple appliances.

MOBILE APPLICATION – REAL-TIME ENERGY MONITORING SCREEN





The above screens shows the mobile application interface of the proposed Smart Appliance Electricity Monitoring System. This application is designed to provide users with real-time access to energy consumption data directly on their smartphones. The mobile app connects to the Flask backend using REST APIs and retrieves live data from the IoT-enabled smart plug. In this screen, the user can view the selected appliance, which is a Laptop Charger, along with its real-time electrical parameters. The application displays the power consumption in watts, energy usage for the current day in kilowatt-hours, and the calculated electricity cost based on tariff slabs. These values are continuously updated based on live data received from the smart plug.

The interface also includes control buttons such as Start and Stop, which allow the user to begin or stop real-time monitoring. Additional features like Download CSV, Download PDF, and Download Graph are provided to export data for analysis and reporting purposes.

A graphical representation of power consumption is also displayed in the mobile app, helping users understand usage patterns over time. An accuracy indicator is shown to represent the reliability of the system's calculations.Overall, this mobile application enhances user convenience by enabling real-time monitoring, analysis, and control of electrical appliances from anywhere using a smartphone.

VOICE ASSISTANT (JARVIS MODULE)

The system also includes a voice assistant module (Jarvis) to provide hands-free control and interaction with the application. This module allows users to perform operations such as starting monitoring, stopping monitoring, and controlling appliances using voice commands. The voice assistant is implemented using JavaScript-based speech recognition and text-to-speech technologies. It captures user voice input through the device microphone and converts it into text commands. These commands are processed and sent to the Flask backend through API requests.Based on the received command, the backend performs appropriate actions such as fetching real-time energy data or controlling the smart plug (ON/OFF). The system also provides voice responses to confirm the action performed, improving user interaction and accessibility.This module enhances user convenience by enabling control without manual interaction. It is especially useful in smart home environments where users can manage appliances using simple voice commands.However, the voice assistant depends on microphone access and may require a stable internet connection for accurate speech recognition. Despite these limitations, it improves the usability and modern features of the system.

DAILY COST

Period	Energy (kWh)	Cost (Rs)
2026-04-27	0.0044	0
2026-04-17	0.0011	0
2026-04-16	0.002	0
2026-04-01	0.0112	0
2026-03-31	0	0
2026-03-30	0.006	0
2026-03-26	0	0
2026-03-25	0.0106	0
2026-03-18	0.0255	0

MONTHLYCOST

Period	Energy (kWh)	Cost (Rs)
2026-04	0.0066	0
2026-03	0.0144	0

The system calculates and displays electricity cost based on real-time energy consumption using predefined tariff slabs.

Daily Cost: Represents electricity cost for the current day
Monthly Cost: Represents accumulated cost over a month
These values help users track their energy expenses and manage usage effectively.

4.3 WORKING MODEL

Step-by-Step Execution

The working model of the system describes the complete process from data collection to user interaction.

1. The electrical appliance is connected to the smart plug.
2. The smart plug is powered ON and connected to a Wi-Fi network.
3. The Flask backend server is started on the system.
4. The backend connects to the smart plug using the Kasa Python library.
5. Real-time energy data such as power and energy consumption is fetched.
6. The collected data is processed and validated.
7. Electricity cost is calculated using predefined tariff slabs.
8. The processed data is stored in the SQLite database.
9. REST APIs are used to send data to the frontend.
10. The user views real-time data on the web dashboard or mobile application.
11. The user can control the appliance by sending ON/OFF commands through the application.

This step-by-step process ensures continuous monitoring, accurate cost calculation, and efficient control of appliances.

4.4 RESULTS & OUTPUTS

The proposed A Tariff Aware Appliance Level Electricity Cost Prediction System successfully generates accurate and real-time outputs based on energy consumption data collected from IoT-enabled smart plugs. The system displays results through both web and mobile interfaces, ensuring easy access and user convenience. The primary output of the system is real-time power consumption, which is measured in watts (W). This value is continuously updated based on the appliance's usage and helps users understand instantaneous energy consumption. The system also calculates daily energy consumption in kilowatt-hours (kWh), which represents the total energy used by the appliance in a day. This helps users track short-term energy usage effectively.In addition to energy monitoring, the system performs electricity cost calculation using predefined tariff slabs. The calculated cost is displayed in Indian Rupees (₹) and is updated dynamically based on real-time consumption. This provides users with a clear understanding of their electricity expenses.The system generates monthly cost outputs, which

represent the accumulated electricity cost over a period of time. This helps users analyze long-term usage patterns and manage their electricity budget efficiently.

Graphical outputs are also generated in the form of line graphs, which display variations in power consumption over time. These graphs help users visualize usage trends and identify peak consumption periods.The system provides options to download reports in CSV and PDF formats. These reports include detailed energy usage, cost data, and historical records, which are useful for analysis and documentation purposes. Another important output is the top energy-consuming appliance identification, which helps users identify devices that consume the most power. This feature supports better energy management and reduces unnecessary consumption. The mobile application displays all outputs in a user-friendly interface, including power, energy, cost, and graphs. It also allows users to control appliances using ON/OFF commands. Overall, the system outputs are accurate, real-time, and informative, enabling users to monitor energy consumption, analyze usage patterns, and reduce electricity costs effectively.

OUTPUT TYPE	**DESCRIPTION**
Power (W)	Real-time power consumption
Energy (kWh)	Daily energy usage
Cost (₹)	Calculated electricity cost
Monthly Cost	Total cost over time
Graphs	Visual energy trends
Reports	PDF & CSV download
Top Consumer	Highest energy device

System Output Parameters and Description

4.5 TESTING

Testing is carried out to ensure that all modules of the system operate correctly, efficiently, and without errors. Different types of testing are performed to validate the functionality, integration, and overall performance of the system.

FUNCTIONAL TESTING

Functional testing is conducted to verify whether each module works as expected. The system modules such as real-time energy monitoring, energy history retrieval, top consumer identification, ranking system, ON/OFF control of appliances using the IoT smart plug, and PDF bill generation are tested individually. The results confirm that all modules function correctly and produce accurate outputs without any errors.

UNIT TESTING

Unit testing is performed to test individual components of the system separately. Key functions such as energy fetching, tariff calculation, database storage, API response handling, and device control operations are tested independently. Each function performs as expected and returns correct results, indicating that the core components of the system are reliable and stable.

Module	Function Tested	Result
Energy Fetch	get_energy()	Pass
Tariff Calculation	tariff()	Pass
Database Insert	store_data()	Pass
API Response	Flask routes	Pass
Device Control	ON/OFF command	Pass

Unit Testing Results

INTEGRATION TESTING

Integration testing is conducted to ensure that all modules of the system work together seamlessly. The communication between the Flask backend and the IoT device, the interaction between Flask and the SQLite database, the connection between the mobile application and Flask APIs, and the integration of the energy calculation module with the IoT device are all tested. The results show that all components are successfully integrated and operate together in real time without any issues.

SYSTEM TESTING

System testing is carried out to validate the complete workflow of the application. The entire process, starting from the user selecting an appliance in the mobile application to fetching energy data from the smart plug, processing and storing the data in the database, and sending the response back to the user interface, is tested. The system also verifies the ON/OFF control functionality of appliances. The results indicate that the system performs all operations successfully in real time.

TEST CASES

Test case validation is also performed to ensure that different user actions produce the expected outputs. Various scenarios such as selecting an appliance, sending ON/OFF commands, viewing energy history, accessing top consumer data, and generating PDF reports are tested. All test cases pass successfully, confirming that the system behaves as expected under different conditions.

Test Case ID	Input	Expected Output	Result
TC1	Select appliance	Show real-time power data	Pass
TC2	ON command	Appliance turns ON	Pass
TC3	OFF command	Appliance turns OFF	Pass
TC4	View history	Display energy data	Pass
TC5	Top consumer API	Highest energy appliance	Pass
TC6	PDF download	Electricity bill generated	Pass

Test Cases and Results

ON/OFF CONTROL TESTING

ON/OFF control testing is specifically conducted to verify the operation of the IoT smart plug. The results show that the device responds correctly to ON and OFF commands, with minimal response time. The control functionality works efficiently through the mobile application using Flask APIs, ensuring smooth remote operation.

API TESTING

API testing is performed using browser tools and API testing platforms to validate all backend endpoints. APIs such as appliance listing, real-time data retrieval, history data access, energy ranking, and live power monitoring are tested. All APIs return correct JSON responses without errors, confirming the reliability of the backend communication.

4.6 PERFORMANCE ANALYSIS

The performance of the system is evaluated based on response time, accuracy, and overall efficiency. The API response time is observed to be within 1 to 2 seconds, while the IoT device response time ranges between 2 to 3 seconds. Database updates occur instantly, and the mobile application reflects data changes in real time. These results indicate that the system provides fast and responsive performance suitable for real-time applications.

ACCURACY

Accuracy is measured by comparing the energy readings obtained from the smart plug with the values processed by the system. The accuracy is calculated using the ratio of the energy measured by the device to the energy computed by the system, expressed as a percentage. The observations show that the energy readings are highly consistent, with only minor variations occurring due to network delays. Overall, the system demonstrates high accuracy and reliability in energy monitoring and cost calculation.

Formula:

$$\textbf{Accuracy (\%)} = \frac{\textbf{Tapo Energy}}{\textbf{System Energy}} \times 100$$

Efficiency

The efficiency of the system is reflected in its ability to provide real-time monitoring, reduce electricity wastage, and enable quick control of appliances. The system enhances user awareness of energy usage and helps in making informed decisions to optimize consumption.

COMPARISON WITH EXISTING SYSTEM

FEATURE	EXISTING SYSTEM	PROPOSED SYSTEM
Monitoring	Total only	Appliance-level
Real-time	No	Yes
Cost Calculation	Manual	Automatic
Control	No	Yes
Mobile Access	No	Yes

CHAPTER V
CONCLUSION AND FUTURE WORK

5.1 CONCLUSION

The Tariff Aware Appliance level electricity cost prediction system has been successfully designed and implemented to monitor and analyze the energy consumption of household appliances. The system uses IoT-enabled smart plugs to collect real-time energy data and display it through a user-friendly web and mobile interface. The system provides appliance-level monitoring, allowing users to track power consumption and calculate electricity cost based on slab-based tariff rates. It also enables identification of high power-consuming devices, helping to reduce electricity wastage and optimize energy usage.In addition to monitoring, the system supports ON/OFF control of appliances, allowing users to remotely manage devices through the mobile application using Flask APIs.The system is efficient, reliable, and suitable for smart home applications. It demonstrates how IoT technology can be effectively used for real-time energy monitoring, cost analysis, and appliance control. The system currently focuses on plug-based appliances and can be extended to non-pluggable devices using smart switches.

5.2 FUTURE WORK

The proposed system can be further enhanced with the following features:

- Integration of Artificial Intelligence for energy prediction and analysis
- Extension of the system to support non-pluggable devices such as ceiling fans and lights using smart switches

APPENDICES

SOURCE CODE – app.py

Code:

```
from flask import Flask, render_template, send_file, request, jsonify
import os
import asyncio
from kasa import Discover, Credentials
from kasa.module import Module
import sqlite3
from datetime import datetime
# PDF imports
from reportlab.platypus import SimpleDocTemplate, Paragraph, Spacer
from reportlab.lib.styles import getSampleStyleSheet
from reportlab.lib.pagesizes import letter
app = Flask(__name__)
# ----------------------------
# Appliance mapping
# ----------------------------
appliance_map = {
    "Appliance1": "Geyser",
    "Appliance2": "Refrigerator",
    "Appliance3": "Microwave",
    "Appliance4": "InductionCooktop",
    "Appliance5": "WashingMachine",
    "Appliance6": "Television",
    "Appliance7": "WaterPurifier",
    "Appliance8": "ElectricKettle",
    "Appliance9": "LaptopCharger",
    "Appliance10": "TableFan",
    "Appliance11": "IronBox",
    "Appliance12": "RiceCooker",
    "Appliance13": "MixerGrinder",
    "Appliance14": "Toaster",
    "Appliance15": "CoffeeMaker",
    "Appliance16": "DesktopPC",
    "Appliance17": "WiFiRouter",
    "Appliance18": "PhoneCharger",
    "Appliance19": "AirPurifier",
    "Appliance20": "VacuumCleaner",
    "Appliance21": "WaterMotor"
}
appliance_images = {
    "Geyser": "mode_heat",
    "Refrigerator": "kitchen",
    "Microwave": "microwave",
    "InductionCooktop": "soup_kitchen",
    "WashingMachine": "local_laundry_service",
    "Television": "tv",
    "WaterPurifier": "water_drop",
    "ElectricKettle": "emoji_food_beverage",
```

```
    "LaptopCharger": "laptop_chromebook",
    "TableFan": "mode_fan",
    "IronBox": "iron",
    "RiceCooker": "rice_bowl",
    "MixerGrinder": "blender",
    "Toaster": "toaster",
    "CoffeeMaker": "coffee_maker",
    "DesktopPC": "desktop_windows",
    "WiFiRouter": "router",
    "PhoneCharger": "smartphone",
    "AirPurifier": "air",
    "VacuumCleaner": "vacuum",
    "WaterMotor": "water"}
def ensure_realtime_schema(cursor):
    cursor.execute("""
    CREATE TABLE IF NOT EXISTS realtime_energy(
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        timestamp TEXT,
        appliance TEXT,
        power REAL,
        today_energy REAL,
        month_energy REAL,
        today_cost REAL,
        month_cost REAL
    )   """)
    cursor.execute("PRAGMA table_info(realtime_energy)")
    columns = [row[1] for row in cursor.fetchall()]
    if "appliance" not in columns:
        cursor.execute("ALTER TABLE realtime_energy ADD COLUMN appliance TEXT DEFAULT
'Unknown'")
    # Backfill older appliance ids (e.g., Appliance9) to readable names.
    for appliance_id, appliance_name in appliance_map.items():
        cursor.execute(
            "UPDATE realtime_energy SET appliance = ? WHERE appliance = ?",
            (appliance_name, appliance_id)
        )
def appliance_db_values(appliance):
    appliance_id = (appliance or "").strip()
    appliance_name = appliance_map.get(appliance_id, appliance_id)
    return appliance_id, appliance_name
def get_appliance_items():
    items = []
    for appliance_id, name in appliance_map.items():
        items.append({
            "id": appliance_id,
            "name": name,
            "icon": appliance_images.get(name, "electrical_services")
        })
    return items
def fetch_latest_realtime_row(appliance=None):
    conn = sqlite3.connect("energy.db")
    conn.row_factory = sqlite3.Row
```

```
    cursor = conn.cursor()
    try:
        ensure_realtime_schema(cursor)
        if appliance:
            appliance_id, appliance_name = appliance_db_values(appliance)
            cursor.execute("""
            SELECT timestamp, appliance, power, today_energy, month_energy, today_cost,
month_cost
            FROM realtime_energy
            WHERE appliance IN (?, ?)
            ORDER BY id DESC LIMIT 1
            """, (appliance_id, appliance_name))
        else:
            cursor.execute("""
            SELECT timestamp, appliance, power, today_energy, month_energy, today_cost,
month_cost
            FROM realtime_energy
            ORDER BY id DESC LIMIT 1
            """)
    except sqlite3.OperationalError:
        conn.close()
        return None
    row = cursor.fetchone()
    conn.close()
    if not row:
        return None
    return {
        "timestamp": row["timestamp"],
        "appliance": row["appliance"],
        "power": row["power"],
        "today_energy": row["today_energy"],
        "month_energy": row["month_energy"],
        "today_cost": row["today_cost"],
        "month_cost": row["month_cost"]
    }
def fetch_top_power_appliance():
    conn = sqlite3.connect("energy.db")
    conn.row_factory = sqlite3.Row
    cursor = conn.cursor()

    try:
        ensure_realtime_schema(cursor)
        cursor.execute("""
        SELECT appliance, AVG(power) AS avg_power, MAX(power) AS peak_power
        FROM realtime_energy
        WHERE power IS NOT NULL
        GROUP BY appliance
        ORDER BY avg_power DESC
        LIMIT 1
        """)
        row = cursor.fetchone()
    except sqlite3.OperationalError:
```

```
conn.close()
        return None
    conn.close()
   if not row:
        return None
    appliance_id = row["appliance"]
    return {
        "id": appliance_id,
        "name": appliance_map.get(appliance_id, appliance_id),
        "avg_power": round(float(row["avg_power"] or 0), 2),
        "peak_power": round(float(row["peak_power"] or 0), 2)
    }

def fetch_top_energy_appliance(period="month"):
    metric_col = "month_energy" if period == "month" else "today_energy"
    cost_col = "month_cost" if period == "month" else "today_cost"

    conn = sqlite3.connect("energy.db")
    conn.row_factory = sqlite3.Row
    cursor = conn.cursor()
    try:
        ensure_realtime_schema(cursor)
        # Pick the latest row for each appliance, then compare energy.
        cursor.execute(f"""
        SELECT r.appliance, r.{metric_col} AS energy_kwh, r.{cost_col} AS cost_rs
        FROM realtime_energy r
        INNER JOIN (
            SELECT appliance, MAX(id) AS latest_id
            FROM realtime_energy
            GROUP BY appliance
        ) last_row ON r.id = last_row.latest_id
        WHERE r.{metric_col} IS NOT NULL
        ORDER BY r.{metric_col} DESC
        LIMIT 1
        """)
        row = cursor.fetchone()
    except sqlite3.OperationalError:
        conn.close()
        return None
    conn.close()
    if not row:
        return None
    appliance_id = row["appliance"]
    return {
        "period": period,
        "id": appliance_id,
        "name": appliance_map.get(appliance_id, appliance_id),
        "energy_kwh": round(float(row["energy_kwh"] or 0), 4),
        "cost_rs": round(float(row["cost_rs"] or 0), 2)
    }
def fetch_energy_ranking(period="month"):
    metric_col = "month_energy" if period == "month" else "today_energy"
```

```
cost_col = "month_cost" if period == "month" else "today_cost"
    conn = sqlite3.connect("energy.db")
    conn.row_factory = sqlite3.Row
cursor = conn.cursor()
    try:
        ensure_realtime_schema(cursor)
        cursor.execute(f"""
        SELECT r.appliance, r.{metric_col} AS energy_kwh, r.{cost_col} AS cost_rs
        FROM realtime_energy r
        INNER JOIN (
            SELECT appliance, MAX(id) AS latest_id
            FROM realtime_energy
            GROUP BY appliance
        ) last_row ON r.id = last_row.latest_id
        WHERE r.{metric_col} IS NOT NULL
        ORDER BY r.{metric_col} DESC """)
        rows = cursor.fetchall()
    except sqlite3.OperationalError:
        conn.close()
        return []
    conn.close()
    ranking = []
    for idx, row in enumerate(rows, start=1):
        appliance_id = row["appliance"]
        ranking.append({
            "rank": idx,
            "id": appliance_id,
            "name": appliance_map.get(appliance_id, appliance_id),
            "energy_kwh": round(float(row["energy_kwh"] or 0), 4),
            "cost_rs": round(float(row["cost_rs"] or 0), 2)
        })
    return ranking
# Tariff Function
def tariff(units):
    cost = 0
    if units <= 100:
        cost = 0
    elif units <= 200:
        cost = (units - 100) * 2.25
    elif units <= 400:
        cost = (100 * 2.25) + (units - 200) * 4.5
    elif units <= 500:
        cost = (100 * 2.25) + (200 * 4.5) + (units - 400) * 6
        cost = (100 * 2.25) + (200 * 4.5) + (units - 400) * 6
    elif units <= 600:
        cost = (100 * 2.25) + (200 * 4.5) + (100 * 6) + (units - 500) * 8
    elif units <= 800:
        cost = (100 * 2.25) + (200 * 4.5) + (100 * 6) + (100 * 8) + (units - 600) * 9
    elif units <= 1000:
        cost = (100 * 2.25) + (200 * 4.5) + (100 * 6) + (100 * 8) + (200 * 9) + (units - 800) * 10
    else:
        cost = (100 * 2.25) + (200 * 4.5) + (100 * 6) + (100 * 8) + (200 * 9) + (200 * 10) + (units -
1000) * 11
```

```
return cost
# STORE DATA
# ----------------------------
def store_data(appliance, power, today_energy, month_energy, today_cost, month_cost):
    conn = sqlite3.connect("energy.db")
    cursor = conn.cursor()
    ensure_realtime_schema(cursor)
   _, appliance_name = appliance_db_values(appliance)
    cursor.execute("""
    INSERT INTO realtime_energy
    (timestamp, appliance, power, today_energy, month_energy, today_cost, month_cost)
    VALUES (?, ?, ?, ?, ?, ?, ?)
    """, (
        datetime.now(),
        appliance_name,
        power,
        today_energy,
        month_energy,
        today_cost,
        month_cost
    ))

    conn.commit()
    conn.close()

# ----------------------------
# GET ENERGY FROM TAPO
# ----------------------------
async def get_energy(appliance, ip, email, password):
    creds = Credentials((email or "").strip(), password or "")
    device = await Discover.discover_single((ip or "").strip(), credentials=creds)
    await device.update()

    energy = device.modules.get(Module.Energy)
    if energy is None:
        raise RuntimeError("Energy module not available on this device.")
    power = float(energy.current_consumption or 0)
    energy_today = float(energy.consumption_today or 0)
    energy_month = float(energy.consumption_this_month or 0)
    today_cost = tariff(energy_today)
    month_cost = tariff(energy_month)
    # store data
    store_data(appliance, power, energy_today, energy_month, today_cost, month_cost)
    return {
        "appliance": appliance_map.get(appliance, appliance),
        "power": power,
        "energy": energy_today,
        "month_energy": energy_month,
        "cost": today_cost,
        "month_cost": month_cost
    }
```

```
# ROUTES
# ----------------------------
@app.route("/")
def welcome():
    return render_template("welcome.html")
@app.route("/instructions")
def instructions():
    return render_template("instructions.html")
@app.route("/appliance")
def appliance():
    return render_template("appliance.html")
@app.route("/realtime")
def realtime():
    appliance = request.args.get("appliance", "").strip()
    appliance_name = appliance_map.get(appliance, "")
    return render_template("realtime.html", appliance=appliance, appliance_name=appliance_name)

@app.route("/history")
def history():

    appliance = request.args.get("appliance", "").strip()
    appliance_name = appliance_map.get(appliance, "")
    return render_template("history.html", appliance=appliance, appliance_name=appliance_name)

@app.route("/top-consumer")
def top_consumer():
    return render_template("top_consumer.html")

@app.route("/api/health")
def api_health():
    return jsonify({"status": "ok"})

@app.route("/api/appliances")
def api_appliances():
    return jsonify({"appliances": get_appliance_items()})

@app.route("/api/realtime/latest")
def api_realtime_latest():
    appliance = request.args.get("appliance", "").strip() or None
    latest = fetch_latest_realtime_row(appliance=appliance)
    if latest is None:
        return jsonify({
            "timestamp": None,
            "appliance": appliance_map.get(appliance, appliance) if appliance else None,
            "power": 0,
            "today_energy": 0,
            "month_energy": 0,
            "today_cost": 0,
            "month_cost": 0})
```

```
    return jsonify(latest)
@app.route("/api/power/top")
def api_power_top():
    top = fetch_top_power_appliance()
    if top is None:
        return jsonify({

            "id": None,
            "name": None,
            "avg_power": 0,
            "peak_power": 0
        })
    return jsonify(top)
@app.route("/api/energy/top")
def api_energy_top():

   period = request.args.get("period", "month").strip().lower()
    if period not in {"today", "month"}:
        return jsonify({"error": "period must be today or month"}), 400
    top = fetch_top_energy_appliance(period="today" if period == "today" else "month")
    if top is None:
        return jsonify({
            "period": period,
            "id": None,
            "name": None,
            "energy_kwh": 0,
            "cost_rs": 0
        })
    return jsonify(top)
@app.route("/api/energy/ranking")
def api_energy_ranking():
    period = request.args.get("period", "month").strip().lower()
    if period not in {"today", "month"}:
        return jsonify({"error": "period must be today or month"}), 400

    rows = fetch_energy_ranking(period="today" if period == "today" else "month")
    return jsonify({
        "period": period,
        "rows": rows
    })
@app.route("/api/history")
def api_history():
    appliance = request.args.get("appliance", "").strip()
    period = request.args.get("period", "daily").strip().lower()
    if period not in {"daily", "monthly"}:
        return jsonify({"error": "period must be daily or monthly"}), 400
    if not appliance:
        return jsonify({"error": "appliance is required"}), 400
conn = sqlite3.connect("energy.db")
    conn.row_factory = sqlite3.Row
    cursor = conn.cursor()
    ensure_realtime_schema(cursor)
```

```
 appliance_id, appliance_name = appliance_db_values(appliance)
    if period == "daily":
        cursor.execute("""
        SELECT date(timestamp) as label,
               AVG(today_energy) as energy,
               AVG(today_cost) as cost
        FROM realtime_energy
        WHERE appliance IN (?, ?)
        GROUP BY date(timestamp)
        ORDER BY date(timestamp) DESC
        LIMIT 30
        """, (appliance_id, appliance_name))
    else:
        cursor.execute("""
        SELECT strftime('%Y-%m', timestamp) as label,
               AVG(month_energy) as energy,
               AVG(month_cost) as cost
        FROM realtime_energy
        WHERE appliance IN (?, ?)
        GROUP BY strftime('%Y-%m', timestamp)
        ORDER BY strftime('%Y-%m', timestamp) DESC
        LIMIT 12
        """, (appliance_id, appliance_name))
    rows = cursor.fetchall()
    conn.close()
    history_rows = [{
        "label": row["label"],
        "energy": round(float(row["energy"] or 0), 4),
        "cost": round(float(row["cost"] or 0), 2)
    } for row in rows]
    return jsonify({
        "appliance": {
 "id": appliance,
            "name": appliance_map.get(appliance, appliance)
        },
        "period": period,
        "rows": history_rows
    })

# ----------------------------
# ACCURACY API (FINAL)
# ----------------------------
@app.route("/accuracy_api")

def accuracy_api():
    conn = sqlite3.connect("energy.db")
    cursor = conn.cursor()
    cursor.execute("""
    SELECT today_energy FROM realtime_energy
    ORDER BY id DESC LIMIT 2
    """)
```

```
 rows = cursor.fetchall()

conn.close()
    if len(rows) < 2:
        return jsonify({
            "accuracy": 0,
            "tapo": 0,

            "dashboard": 0
        })
    tapo_energy = rows[0][0]
    dashboard_energy = rows[1][0]

    if dashboard_energy > 0:
        accuracy = (tapo_energy / dashboard_energy) * 100

    else:
        accuracy = 0

    return jsonify({
        "tapo": round(tapo_energy, 4),
        "dashboard": round(dashboard_energy, 4),
        "accuracy": round(accuracy, 2)
    })

# ----------------------------
# LIVE DATA API
# ----------------------------
@app.route("/livepower", methods=["POST"])
def livepower():

    data = request.get_json(silent=True) or {}

    appliance = (data.get("appliance") or "").strip()
    ip = (data.get("ip") or "").strip()
    email = (data.get("email") or "").strip()
    password = data.get("password") or ""

    if not appliance or appliance not in appliance_map:
        return jsonify({"error": "valid appliance is required"}), 400

 if not ip or not email or not password:
        return jsonify({"error": "ip, email, and password are required"}), 400

    try:


 result = asyncio.run(get_energy(appliance, ip, email, password))
        return jsonify(result)
    except Exception as e:
        return jsonify({"error": str(e)}), 500
 # DOWNLOAD PDF
```

```
@app.route("/download_pdf", methods=["POST"])
def download_pdf():
 power = request.form.get("power")
    energy = request.form.get("energy")
    cost = request.form.get("cost")
    month_cost = request.form.get("month_cost")
    file_path = "Electricity_Bill.pdf"
    doc = SimpleDocTemplate(file_path, pagesize=letter)
    styles = getSampleStyleSheet(
    content = []
    content.append(Paragraph("Electricity Bill Report", styles['Title']))
    content.append(Spacer(1, 20))
    content.append(Paragraph(f"Current Power: {power} W", styles['Normal']))
    content.append(Paragraph(f"Today's Energy: {energy} kWh", styles['Normal']))
    content.append(Paragraph(f"Today's Cost: ₹{cost}", styles['Normal']))
    content.append(Paragraph(f"Monthly Cost: ₹{month_cost}", styles['Normal']))
    doc.build(content)
    return send_file(file_path, as_attachment=True)
# RUN SERVER
if __name__ == "__main__":
    # Default to HTTP for local mobile testing; set FLASK_HTTPS=1 when needed.
    use_https_raw = os.environ.get("FLASK_HTTPS", "0").strip().lower()
    use_https = use_https_raw not in {"0", "false", "no", "off"}
    cert_file = os.environ.get("SSL_CERT_FILE", "").strip()
    key_file = os.environ.get("SSL_KEY_FILE", "").strip()
    ssl_context = None
    if use_https:
        if cert_file and key_file and os.path.exists(cert_file) and os.path.exists(key_file):
  ssl_context = (cert_file, key_file)

   else:
            ssl_context = "adhoc"

    protocol = "https" if use_https else "http"
    print(f"Starting server on {protocol}://0.0.0.0:5000")
    print(f"LAN URL: {protocol}://<your-pc-ip>:5000")
    if use_https and ssl_context == "adhoc":
        print("Using temporary self-signed certificate (browser warning is expected).")
        print("Set SSL_CERT_FILE and SSL_KEY_FILE to use a trusted local certificate.")
    app.run(host="0.0.0.0", port=5000, debug=True, ssl_context=ssl_contex
```

REFERENCES

1. IEEE, “IEEE Standard for Smart Energy Monitoring Systems,” IEEE Publications, 2020.
2. P. Sharma et al., “Smart Energy Monitoring System using IoT and Mobile Application,” IJERT, vol. 9, no. 6, 2020.
3. K. Suresh et al., “Real-Time Energy Monitoring using IoT and Cloud Computing,” IJACSA, vol. 12, no. 4, 2021.
4. R. Kumar et al., “IoT-Based Smart Energy Management System for Home Automation,” IJSTR, vol. 11, no. 3, 2022.
5. S. Verma et al., “IoT-Based Electricity Monitoring and Billing System,” IJEC, vol. 11, no. 5, 2022.
6. J. Li et al., “Machine Learning-Based Energy Consumption Prediction in Smart Homes,” IEEE Access, 2023.
7. L. Wang et al., “AI-Based Smart Home Energy Management System,” IEEE Transactions on Smart Grid, 2023.
8. Y. Zhang et al., “Edge Computing-Based Smart Energy Monitoring System,” Journal of Cloud Computing, 2024.
9. M. Ali et al., “Real-Time Smart Energy Metering using IoT and Data Analytics,” International Journal of Energy Research, 2024.
10. Dandan Li et al., “Temporal-Spatial Graph Network for Appliance-Level Prediction,” IEEE TNNLS, 2025.
11. Gupta et al., “IoT-Based Smart Metering System,” International Journal of Engineering Research, 2021.
12. N. Patel et al., “Energy Consumption Monitoring using Smart Devices,” IEEE Conference Proceedings, 2022.
13. S. Reddy et al., “Smart Home Automation with Energy Monitoring,” IJCSIT, 2021.
14. M. Khan et al., “IoT-Based Energy Efficiency System,” Springer Journal of Smart Systems, 2022.
15. T. Nguyen et al., “Real-Time Energy Analytics using IoT,” Elsevier Energy Reports, 2023.
16. R. Singh et al., “Smart Plug-Based Energy Monitoring System,” IEEE Conference, 2022.
17. J. Park et al., “Appliance-Level Energy Monitoring Techniques,” IEEE Access, 2023.
18. H. Chen et al., “Energy Optimization in Smart Homes,” IEEE Smart Grid Journal, 2024.
19. Flask Documentation, https://flask.palletsprojects.com/
20. Python Software Foundation, https://www.python.org/
21. SQLite Documentation, https://www.sqlite.org/
22. Kasa Python Library Documentation, https://python-kasa.readthedocs.io/
23. Android Studio Developer Documentation, https://developer.android.com/
24. TP-Link, “Tapo Smart Plug (P110) User Guide,” https://www.tapo.com/
25. World Bank, “Energy Consumption and Efficiency Reports,” https://www.worldbank.org/